1.1 Introduction to package fsimp

The `fsimp` package provides a 'fullsimp' simplification function for expressions. This code was adapted from David Scherfgen's dssimp.mac code posted on the Maxima discussion list.

This code implements a simplification routine for Maxima CAS expressions using the core simplification routines for expressions invovling trigonometric, logarithmic, exponential, polynomial and other functions.

To install the package with mxpm, run `mxpm install --path` on a copy of the repository. Without mxpm, copy the repository into a directory under `~/.maxima`, which Maxima 5.47 and later search, or add its directory to `file_search_maxima` and `file_search_lisp`, or load it by its full path. Then

```
load("fullsimp");
```

loads the package. `load("fsimp")` also works. To simplify an expression `expr` call

```
fullsimp(expr);
```

The functions, variables and rules the package defines all start with `fs`, apart from `fullsimp` itself, so loading it leaves the names a user works with alone. An input with `gamma` or a factorial loads Maxima's `gamma_simp` share package, which defines functions of its own in the session; excluding its routines, as in `fullsimp(e, gamma_simp, factorial_simp)`, keeps it from loading.

Loading the package also undoes one change that Maxima itself makes. Maxima's `trigsimp`, which `fullsimp` uses, declares `a` as a pattern variable with `matchdeclare` when it first loads, so that `a` matches any expression in rules the user later defines with `defrule`, `tellsimp` or `let`. Loading `fsimp.mac` loads `trigsimp` at once and removes that declaration, unless `a` already had one. The rules of `trigsimp` are compiled by then, so `trigsimp` works as before. A user who wants `a` as a pattern variable can declare it again with `matchdeclare`.

The algorithm begins by searching through an expression for e.g. trig, log, exp, etc. and choosing a set of core simplification routines. Then it runs a poor-man's kind of simulated annealing routine. It's poor-man's because it's not really simulated annealing, but while it cycles through various core simplification routines (several times) it allows the intermediate expression to become slightly larger if it ultimately becomes smaller. The output is always the smallest size expression, where the size is computed by the lisp `conssize` function plus the string length of the expression, with each variable, and each name of a function Maxima has no simplifier for (such as an undefined `f`), counted as one character. Renaming variables or such functions therefore leaves the size of every expression unchanged.

By default the search keeps a small pool of candidates instead of a single one (a beam search). The input expression always stays in the pool, so every routine is applied to the original as well as to the intermediate results, next to the cheapest candidates found so far and one candidate that is allowed to be larger. Each round applies every routine to every new member of the pool, and the search stops when a round adds nothing new. The result is never larger than what any single routine produces from the input. The original single-candidate sweep is still available. See `fs_search`, `fs_beam_width` and `fs_max_rounds`.

When the expression is a sum or product of parts that share no variables, such as a polynomial in `a` and `b` plus a trigonometric expression in `x`, each part is simplified on its own, so a routine suited to one part never runs on the others. See `fs_split`.

Some core routines, such as `radcan`, `rootscontract` and `logcontract`, rewrite expressions in ways that are only valid for part of the complex plane. For example `radcan` turns `sqrt(x^2+2*x+1)` into `x+1`, which is wrong for `x < -1`. So `fullsimp` checks every candidate against the input expression numerically, at a fixed set of test points, and discards any candidate that disagrees at a point. The test points cover positive and negative reals and all four complex quadrants, and they respect the `assume` database and `declare`d features: after `assume(x > -1)`, `fullsimp(sqrt(x^2+2*x+1))` returns `x+1`. See `fs_check`.

The rtest files `rtest_fsimp1.mac` to `rtest_fsimp3.mac`, `rtest_fsimp_check.mac`, `rtest_fsimp_beam.mac` and `rtest_fsimp_split.mac` test the package. `tests/run_rtests.sh` runs each file in its own Maxima process and reports pass counts and timings. `tests/fsimp_audit_rtests.mac` checks that every expected result in the rtest files equals its input numerically, independently of `fs_check`. `tests/fsimp_compare_single.mac` checks that `fullsimp` is never costlier than any single routine applied to the input, and `tests/fsimp_check_collisions.mac` that binding a variable outside `fullsimp` never changes its result.

E. Majzoub.

1.2 Functions and Variables for fsimp

`fsdebug` [Variable]

Setting the variable `fsdebug` to 1 will allow the user to see the fsimp function operations, list selection of simplification routines, etc. With `fsdebug` at 1 or more, each candidate rejected by the equivalence check is printed together with the routine that produced it, and a note is printed when the assumptions leave too few test points, in which case the check samples without them. fullsimp also reports when it splits an expression into independent groups (see `fs_split`), and when the combined result fails the check, in which case it simplifies the expression whole. At 2 or more, the beam search also prints the pool sizes after each round and why it stopped.

`fs_check` [Variable]

Default value: `true`

When `fs_check` is `true`, `fullsimp` only returns expressions equal in value to its input. It evaluates the input and each candidate at the same test points and rejects a candidate that disagrees at any of them. The test points are chosen so that every pair of variables takes every combination of sign and quadrant, and they follow the `assume` database: a variable with `assume(x > 0)` is only sampled at positive values, so rewrites that hold for positive `x` are accepted. Functions with no numeric definition, such as an undefined `f(x)` or a quoted `'f(x)`, are replaced by fixed stand-in functions for the comparison. Values follow Maxima's own conventions, so an odd root of a negative number is its real root: `(-8)^(1/3)` is `-2`, and a rewrite of `%e^(log(x)/3)` to `x^(1/3)` is rejected because the two differ for negative `x`.

When the input cannot be evaluated numerically at all, for example because it contains an unevaluated integral, only candidates produced by routines that preserve value everywhere (such as `ratsimp`, `factor` and `trigsimp`) are accepted. Routines added through `fs_custom_simp` never count as preserving value.

When fewer than half the test points can be drawn under the assumptions, for example because they are contradictory or hard to satisfy, the check uses points that ignore the assumptions instead.

The check is numeric, so it cannot see differences confined to isolated points. `(x^2-1)/(x-1)` still simplifies to `x+1`, as it does elsewhere in Maxima.

Setting `fs_check` to `false` turns the check off and gives the results of earlier versions.

`fs_search` [Variable]

Default value: `beam`

Selects the search `fullsimp` uses. With `beam`, it runs the beam search described in the introduction, bounded by `fs_beam_width` and `fs_max_rounds`. With `sweep`, it runs the search of earlier versions: two passes over the routine list, keeping a single current expression. Any other value is an error.

`fs_beam_width` [Variable]

Default value: `4`

The number of candidates the beam search keeps in its pool: the input, `fs_beam_width - 2` of the cheapest candidates found so far, and one candidate up to 80% larger than the cheapest. With a width of 2 the pool holds the input and the cheapest candidate. With a width of 1 it holds only the cheapest candidate, so the search climbs greedily from the input. Larger widths explore more alternatives and take longer. The value must be a positive integer.

`fs_max_rounds` [Variable]

Default value: `4`

The largest number of rounds the beam search runs. It usually stops earlier, as soon as a round adds no new candidate to the pool. The value must be a positive integer.

`fs_split` [Variable]

Default value: `true`

When `fs_split` is `true` and the expression is a sum or product whose terms fall into two or more groups that share no variables, `fullsimp` simplifies each group on its own and combines the results. Terms with no variables, such as a constant `1`, may go with any group: each group either keeps its own result or is simplified again with all the constants added, and a group containing a trigonometric or hyperbolic function also tries `+1` and `-1`, with the rest of the constant added back. The cheapest combination is kept. So `1 - sin(x)^2 + tan(y)^2 + 1` gives `cos(x)^2 + sec(y)^2`, and `cosh(z)^2 - sin(x)^2` gives `sinh(z)^2 + cos(x)^2`. A group that simplifies to a constant joins the constants. Afterwards `logcontract` and `rootscontract` are tried on the combination, which merges forms like `log(a) + log(b)` across groups.

Splitting keeps large independent parts from slowing each other down: `fullsimp(expand((a+b+c+d+u)^10) + sin(x)^2 + cos(x)^2)` returns `(a+b+c+d+u)^10 + 1` in well under a second. For the same reason a group larger than 1000 (in the size `fullsimp` uses) takes no shift. The price is that factorisations across groups are not found: `(x+1)^2 - y^2` is not turned into `(x+1-y)*(x+1+y)`.

Setting `fs_split` to `false` simplifies the expression as a whole.

`fullsimp` (`expr`,[*simp1,simp2,...*]) [Function]

The `fullsimp` function takes an input expression `expr` and attempts to find a simpler form using the builtin simplification routines in core Maxima. These include: `[resimplify, expand, combine, radcan, ratsimp, rootscontract, xthru, multthru, factor, sqrtdenest, triglist,exptlist, loglist]`. Additional trigonometric simplifications are applied as well. The full list of simplifications is shown in the variable `simplist` which will be shown if the variable `fsdebug` is set to 1.

There are two ways to manipulate the simplifications in `simplist`. First, the option arguments to `fullsimp`, `simp1, simp2, etc.` are simplification routines the user wants *excluded* from `simplist`.

Second, the user may create a *list* called `fs_custom_simp`, and place any user defined simplification routines. For example the code

```
fs_custom_simp:[lambda([q], block([algebraic : true], ratsimp(q)))]
```

will create a simplification routine where the variable `algebraic` is set to `true`, while the `ratsimp` simplification is applied.

In general, `fullsimp` tries to find the smallest equivalent expression based on the Common Lisp `conssize` and string size of an expression, counting each variable and each undefined function name as one character.

The user may manipulate the order of simplifications through use of the exclusion option and the custom simplification option.

`fullsimp` is a simplifying function and can be added to the command line as shown in the last example below.

```
(%i1) load("fsimp.mac")$
(%i2) fullsimp((1+cos(t))/sin(t));
                                        t
(%o2)                                cot(-)
                                        2
(%i3) fullsimp(cos(x)+%i*sin(x));
                                      %i x
(%o3)                                 %e
(%i4) (cos(t)+1)/((%i*sin(p)-cos(p))*sin(t)),fullsimp;
                                      %i p     t
(%o4)                          - %e     cot(-)
                                            2
```

The equivalence check keeps rewrites that are not valid for all `x` out of the result, unless assumptions make them valid.

```
(%i1) load("fsimp.mac")$
(%i2) fullsimp(sqrt(x^2+2*x+1));
                                       2
(%o2)                        sqrt(x  + 2 x + 1)
(%i3) assume(x>-1);
(%o3)                               [x > - 1]
(%i4) fullsimp(sqrt(x^2+2*x+1));
(%o4)                                x + 1
```

If you wish to exclude particular simplification routines, add them to the arguments of fullsimp as shown below.

```
(%i1) load("fsimp.mac")$
(%i2) fullsimp((1+cos(t))/sin(t),fstrigsimp,trigsimp,trigreduce);
                                    cos(t) + 1
(%o2)                               ----------
                                      sin(t)
```

Appendix A Function and Variable index

fullsimp 4

fs_beam_width 3
fs_check 2
fs_max_rounds 3
fs_search 3
fs_split 3
fsdebug 2